ETH zürich

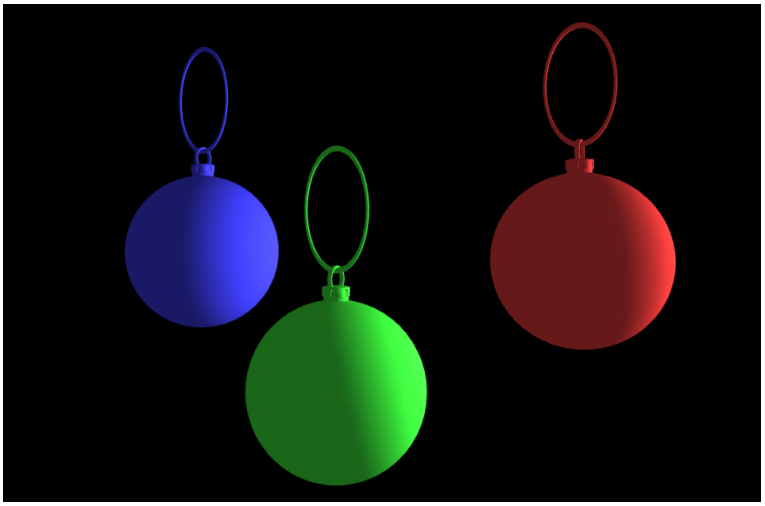
computer graphics laboratory

Prof. Dr. Markus Gross

Visual Computing

Exercise 7 / Week 8: Introduction to WebGL

Hand-out Date: 02 November 2026 · Revised: 04 November 2026



Goals

- Getting acquainted with the WebGL graphics library API.
- Learning about the GL graphics pipeline, shaders, and buffers.
- Understanding how to apply colors and simple transformations.

Resources

Chapter 22 of the script (The Web Graphics Library) covers what you need for this exercise. Section 22.2.4 explains vertex buffers, attributes, indexed drawing and `gl.drawElements` (Tasks 1 and 3a). Transformations for Task 3b and the gl-matrix vector functions for Task 2 are in Section 22.2.5. Section 22.2.6 shows how to activate the shader program with `gl.useProgram`, set uniforms (Task 1b) and compute vertex normals from face normals (Task 2).

Read `readme.md` in the exercise folder before you start. It explains how to open the exercise through a local web server and how to find errors in the browser console. It also includes hints and spoilers for each task.

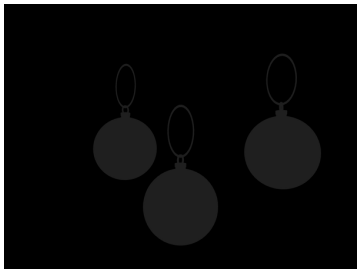
We sometimes improve the exercise after it is published. Before you start, pull the latest version of the repository. If you have already started, you can keep working in your own copy.

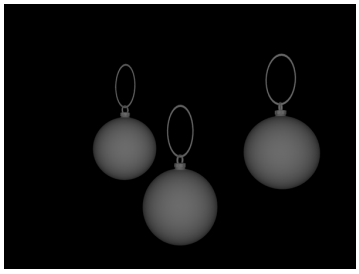
Tasks

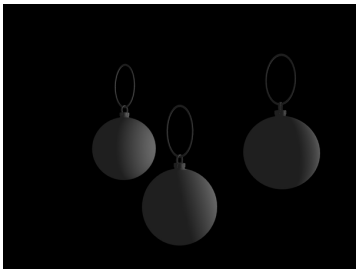
1. Mesh Setup and Initialization

In this exercise, we will set up a mesh (Christmas decoration) for rendering. All code needed for this project is contained in `main.js` and the shader files `vertexShader.js` and `fragmentShader.js`.
First, get acquainted with the program structure. Then, have a look at the shader code. Note which attributes and uniforms are used in the fragment and vertex shader. `main.js` runs on the CPU and the shaders run on the GPU, so they can't see each other's variables. In this task you connect them: attributes are shader inputs with a different value for every vertex, and uniforms are shader inputs with one value for the whole mesh. Once you have familiarized yourself with the code structure, complete the following two tasks. Drawing is already set up, so you see dark, flat baubles from the start. The tasks are not in order in `main.js`, so search for "Task 1a" to find the start:

- **Binding the Data**: Locate the part of the code marked as "Task 1a". The buffers for the vertex positions and the mesh faces are already done. Read the position block line by line, then add the same block for the normals, so that `mesh.vertexNormals` reaches the attribute `aVertexNormal` (the procedure is covered in Section 22.2.4 of the script). When implemented correctly, the baubles become bright and shaded. These normals come from the OBJ file. The light still sits at the camera.
- **Uniform Variables**: Locate the part of the code marked as "Task 1b". The shader program, the two matrices and the draw call are already done. Look at the three uniform inputs of the vertex shader and at how the two matrices are set. Then set the third uniform, `uLightPos`, from `main.js` (the procedure is covered in Section 22.2.6 of the script). When implemented correctly, the light comes from the right. The picture gets darker, because most of each bauble is now in the shadow.


Start

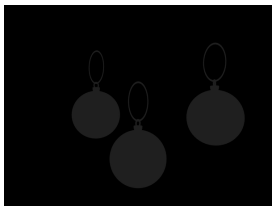

After 1a

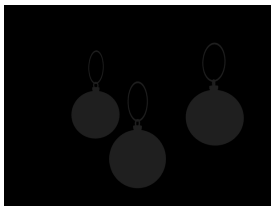

After 1b

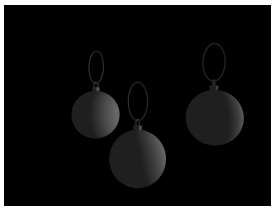
2. Normals for Lighting

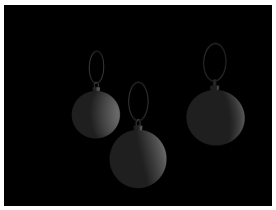
In this part, we will add simple lighting in order to see the details of the 3D mesh. In the vertex shader, a simple diffuse lighting method is already implemented. This code, however, requires per-vertex normals to calculate the lighting. A triangle mesh only has well-defined normals per face, so we approximate the vertex normals from the face normals as follows (see code marked as "Task 2"). So far you have used the normals from the OBJ file. Now you compute your own: at the start of the part marked as "Task 2", remove the slashes in front of the loop that sets all normals to zero. The baubles turn flat and dark again.

- **Face normals**: First, calculate the face normal for each triangle based on the vertex positions that correspond to each face by following the steps given as comments in the code (the gl-matrix vector functions are covered in Section 22.2.5 of the script). The picture doesn't change yet, because the vertex normals are still zero.
- **Vertex Normals**: Next, approximate the vertex normals by averaging all the face normals around that vertex (the procedure is covered in Section 22.2.6 of the script). The shading appears in this step.
- **Normalization**: In a final step, we iterate through the calculated vertex normals and normalize them to unit length. The picture doesn't change in this step, because the vertex shader also normalizes the normals. The baubles stay grey and shaded, with the light coming from the right.


Start of Task 2


After Face normals


After Vertex Normals


After Normalization

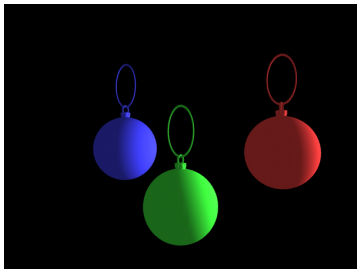
3. Color and Rotation

In this part, we will color the mesh and rotate it. Note that the vertex color is currently set to `vec4(0.3, 0.3, 0.3, 1.0)` in the vertex shader. We want to extend the vertex shader to take a custom vertex color as attribute, which should be precomputed in the main code as follows:

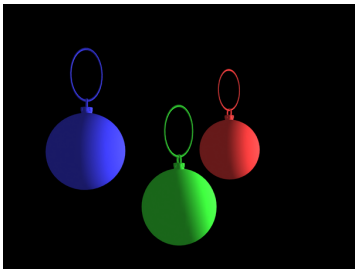
- **Coloring**: In the code marked as "Task 3a", add a new array `mesh.vertexColors` to the mesh for storing the colors. Each color is a `vec4` (r, g, b, a), so `mesh.vertexColors` needs 4 numbers per vertex, one more than the positions and normals. Instead of giving the same color to all the Christmas baubles, color each of them differently (Hint: the mesh can be split up into three regions

that separate the baubles using two planes going through $y = -8$ and $y = 9$). Then, add a new attribute `aVertexColor` to the vertex shader (also marked as "Task 3a" there) and extend the code in "Task 1a" to create a buffer for the new vertex colors and bind them analogously to the other attributes used in the vertex shader (see the procedure in 1a and Section 22.2.4 of the script). Finally, do not forget to use the new `aVertexColor` instead of the constant color inside the vertex shader. If implemented correctly, the three baubles will appear in different colors, similar to the image on the first page.

- **Rotation**: In order to rotate the mesh, apply a rotation around the z-axis to the `modelViewMatrix` in the code marked as "Task 3b" (rotations are covered in Section 22.2.5 of the script). When implemented correctly, the baubles turn around their vertical axis, and the light stays on the right.



After 3a



After 3b